DSA PRACTICE WORKBOOK

Four Traversal Problems: Decision Practice Workbook

A 75-minute mixed BFS and DFS practice set with original prompts, worked traces, invariants, tests, near misses, and an explanation rubric.

Choose traversal from the guarantee the prompt needs, keep state separate from the answer, and prove correctness across four representative graph problems.

VERSION 1.0

FORMAT GUIDE + WORKSHEET

STATUS LEARNER READY

DSA / TRAVERSAL-DECISION-PRACTICE

COMPLETE THE EXACT WEEK 3 TASK

Four problems, four visible decisions

This workbook turns traversal practice into evidence: choose from the guarantee, name the state, preserve an invariant, test the boundary, and explain the near miss.

TASK OBJECTIVE

Solve one component problem, one shortest unweighted path, one directed dependency-cycle problem, and one path-reconstruction problem. Attempt each prompt before opening its worked review.

01	02	03	04
Classify	**Separate**	**Prove**	**Review**
Name the guarantee the prompt requires before naming BFS or DFS.	Define frontier, discovery, problem state, and final answer independently.	State the invariant and the exact moment a node becomes discovered.	Test boundaries, defend complexity, and name a prompt change that breaks the choice.

The 75-minute practice block

Clock	Activity	Required artifact
0-8	Refresh the decision model	Guarantee, frontier, state, invariant, visited timing
8-20	Problem 1: components	Attempt plus three tests
20-33	Problem 2: minimum edges	Attempt plus queue-layer trace
33-46	Problem 3: dependency cycle	Attempt plus active-path trace
46-61	Problem 4: reconstruct route	Attempt plus parent chain
61-70	Open the worked reviews	Compare decisions, not just final answers
70-75	Save evidence	One repeated weakness and one correction sentence

Evidence to save for every problem

- [ ] Decisive cue and required guarantee.
- [ ] Traversal, frontier, state meanings, and invariant.
- [ ] At least three boundary tests and a complexity explanation.
- [ ] One near miss: the smallest prompt change that would require another method.

DECISION MODEL

Choose from the guarantee, not the keyword

Words such as graph, path, or dependency do not choose the algorithm. The required guarantee and the state carried through the traversal do.

LAYER GUARANTEE

BFS

Use a queue when equal-cost edges must be explored by layer: minimum edges, minimum steps, nearest target, or level order.

BRANCH GUARANTEE

DFS

Use a stack or recursion when the central job is to finish a component, explore a branch, or expose active-path state.

Prompt requirement	Default tool	State that makes it correct	Disqualifying cue
Explore every node in one component	DFS or BFS	Global discovered set	If minimum distance is required, order now matters
Minimum edges or steps	BFS	Distance at discovery; queue by layer	Unequal edge costs break plain BFS
Directed cycle on the active dependency path	3-color DFS	Unseen / visiting / done	One visited set cannot identify a back edge
Return an actual shortest route	BFS + parent	Parent fixed on first discovery	DFS proves reachability, not minimum edges
Nonnegative weighted shortest path	Weight-aware search	Best-known cost and priority frontier	Plain BFS / DFS is not enough

TRUTHFUL NUANCE

BFS and DFS are both correct for basic reachability and component discovery. Choose deliberately, then explain the real trade-off: queue width, stack depth, path-local state, or a required layer guarantee.

Separate these four layers

01

Frontier

Queue, stack, or recursion: what is processed next.

02

Discovery

What has already entered the search and cannot be added again.

03

Problem state

Distance, color, component ID, indegree, or other meaning.

04

Answer

Count, boolean, parent chain, path, or requested evidence.

PROBLEM 1 / ATTEMPT FIRST

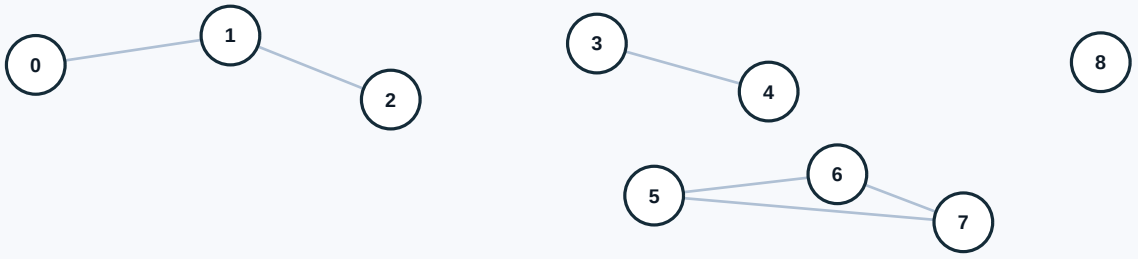
Count connected components

This is a disconnected undirected graph. The result must include isolated nodes and every component, not only the region reachable from node 0.

PROMPT

Given n = 9 nodes labeled 0 through 8 and undirected edges {0-1, 1-2, 3-4, 5-6, 6-7, 7-5}, return the number of connected components.

UNDIRECTED COMMUNICATION GRAPH



REPRESENTATION

Input contract

Adjacency-list graph; an isolated node has an empty neighbor list; edges may appear in both directions.

OUTPUT

Return contract

One integer: the number of disjoint connected components across all n nodes.

Write these decisions before coding

- [ ] What prompt cue makes traversal appropriate, and does BFS versus DFS affect correctness here?
- [ ] When is a node marked discovered: before it enters the frontier or after it leaves?
- [ ] Why is an outer loop over every node necessary?
- [ ] What remains true when one complete traversal finishes?
- [ ] Test an isolated node, no edges, one large component, and the triangle 5-6-7.

ATTEMPT BOUNDARY

Spend up to 12 minutes. Save your count, pseudocode or code, invariant, three tests, complexity, and one near-miss before turning the page.

PROBLEM 1 / WORKED REVIEW

Finish one component before counting the next

An iterative DFS is used here to make stack depth explicit. BFS would also be correct because this prompt asks for component membership, not minimum distance.

01
Cue
Count every disconnected region, including isolated nodes.

02
Choice
Scan all nodes; start iterative DFS from each unseen node.

03
State
A global discovered set survives between component traversals.

04
Invariant
Each completed start marks exactly one full component.

```
count = 0
discovered = empty set
for node in 0 .. n - 1:
    if node in discovered: continue
    count = count + 1
    stack = [node]; add node to discovered
    while stack is not empty:
        current = stack.pop()
        for next in neighbors[current]:
            if next not in discovered:
                add next to discovered; stack.push(next)
return count
```

Outer-loop start	Newly finished component	Discovered after traversal	Count
0	{0, 1, 2}	{0, 1, 2}	1
3	{3, 4}	{0, 1, 2, 3, 4}	2
5	{5, 6, 7}	{0, 1, 2, 3, 4, 5, 6, 7}	3
8	{8}	All nine nodes	4

WHAT FAILS
Near miss
One DFS from node 0 returns one component and silently misses nodes 3 through 8. If the prompt changes to minimum edges, DFS no longer supplies the required guarantee.

WHY IT HOLDS
Boundary proof
Mark on push so duplicate edges and the 5-6-7 cycle cannot add the same node repeatedly. The isolated node 8 still starts and completes a component.

COMPLEXITY

With adjacency lists, every node is discovered once and every stored edge is scanned: O(V + E) time and O(V) discovered/frontier space.

PROBLEM 2 / ATTEMPT FIRST

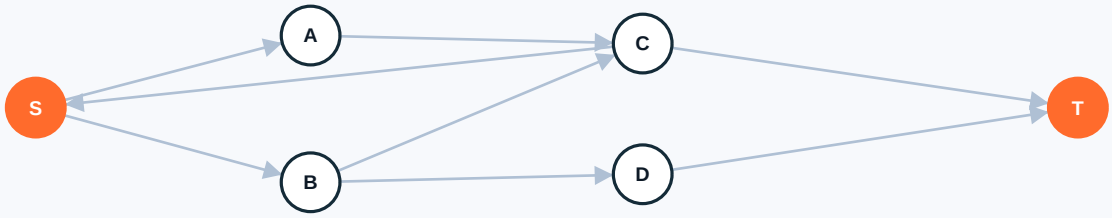
Find the minimum number of edges

The graph is directed and every edge has equal cost. Duplicate edges and a cycle are present, so discovery timing is part of correctness and efficiency.

PROMPT

Adjacency: S -> [A, B, B], A -> [C], B -> [C, D], C -> [S, T], D -> [T], T -> []. Return the minimum number of edges from S to T, 0 when source equals target, and -1 when unreachable.

DIRECTED EQUAL-COST GRAPH; THE DUPLICATE S -> B INPUT EDGE IS SHOWN ONCE



MINIMUM

Required guarantee

The returned distance must be the smallest number of directed equal-cost edges from source to target.

DUPLICATES + CYCLE

Noise to handle

S points to B twice and C points back to S. Neither should duplicate useful work.

Before coding

- [ ] Name the cue that rules out a plain DFS answer.
- [ ] Define what the queue stores and what distance means at discovery.
- [ ] Choose mark-on-enqueue or mark-on-dequeue and defend the timing.
- [ ] Write the layer invariant and the unreachable return contract.
- [ ] Test source equals target, direct edge, duplicate edge, cycle, and unreachable target.

ATTEMPT BOUNDARY

Spend up to 13 minutes. Draw the queue by layer and record the distance assigned to each node on first discovery.

PROBLEM 2 / WORKED REVIEW

First discovery is shortest only because edges cost the same

BFS processes equal-cost edges in layers. Distance can double as the discovery marker, preventing duplicate queue entries and preserving the first shortest distance.

01	02	03	04
Cue	**Choice**	**State**	**Invariant**
Minimum number of equal-cost directed edges.	BFS from S with distance assigned before enqueue.	Queue frontier plus distance map; no separate visited set needed.	Queued nodes have final shortest distance and leave in nondecreasing distance.

```
distance = {source: 0}
queue = [source]
while queue is not empty:
    node = queue.pop_left()
    if node == target: return distance[node]
    for next in neighbors[node]:
        if next in distance: continue
        distance[next] = distance[node] + 1
        queue.push_right(next)
return -1
```

Layer	First discoveries	Frontier before layer	Meaning
0	S = 0	[S]	Source contract
1	A = 1, B = 1	[A, B]	Duplicate S -> B is ignored
2	C = 2, D = 2	[C, D]	C is not enqueued twice
3	T = 3	[T]	First T discovery proves minimum 3

WRONG GUARANTEE

Near miss

DFS may return a valid but longer path first. Marking only on dequeue lets duplicate edges add the same node repeatedly and can destabilize later state.

SWITCH METHOD

Prompt change

If edges carry unequal nonnegative costs, minimum edges is not minimum cost. Use a cost-aware method such as Dijkstra instead of plain BFS.

COMPLEXITY

Each discovered vertex enters the queue once; every adjacency entry, including duplicates, is scanned once: O(V + E) time and O(V) space.

PROBLEM 3 / ATTEMPT FIRST

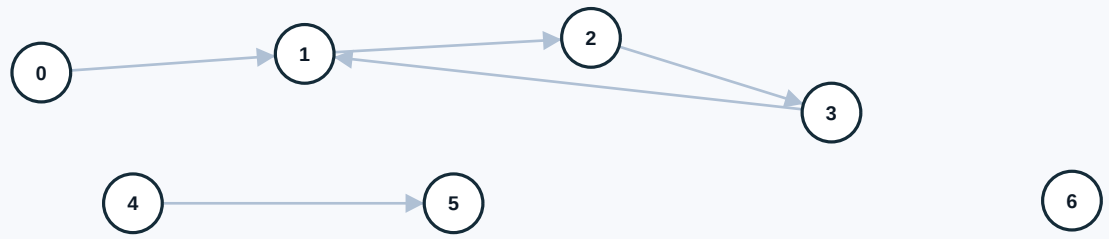
Detect a directed dependency cycle

Edge direction is part of the contract: u -> v means u must finish before v. A cycle means at least one chain of work can never become complete.

PROMPT

Tasks are 0 through 6 with dependencies 0 -> 1, 1 -> 2, 2 -> 3, 3 -> 1, and 4 -> 5. Node 6 is isolated. Return whether every task can finish; if not, identify one blocking cycle.

DIRECTED PREREQUISITE GRAPH



COMPLETED STATE

Global fact

A node may have been seen in an earlier, completely finished branch.

VISITING STATE

Path-local fact

A node may still be active in the current dependency chain.

Define the evidence before coding

- [ ] Choose 3-color DFS or Kahn's indegree BFS and state why it exposes the evidence you need.
- [ ] If using DFS, define unseen, visiting, and done precisely.
- [ ] Name the exact edge event that proves a directed cycle.
- [ ] Handle a self-loop, a safe edge to a completed node, disconnected work, and an isolated node.
- [ ] State the dependency edge direction before interpreting any output.

ATTEMPT BOUNDARY

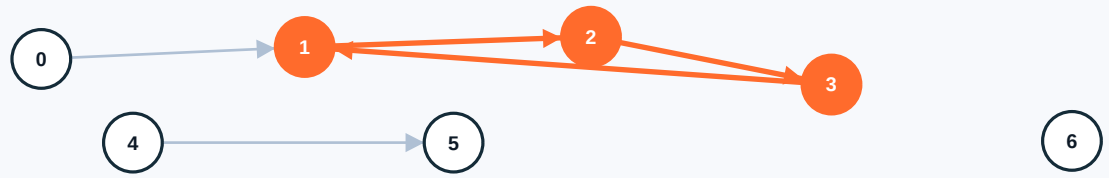
Spend up to 13 minutes. Save the color transition trace for the first cycle proof; do not rely on a single visited set.

PROBLEM 3 / WORKED REVIEW

A back edge to the active path proves the cycle

Three-color DFS distinguishes a safe edge to completed work from an edge that re-enters the dependency chain currently being explored.

CYCLE EVIDENCE: 1 -> 2 -> 3 -> 1



01
Enter
Set the node to visiting when its DFS frame opens.

02
Explore
Recurse on unseen work; an edge to visiting work proves a cycle.

03
Finish
Mark done only after every outgoing edge is safe.

04
Scan
Start from every unseen node to include disconnected work.

```
color = all UNSEEN; active = []; position = {}
visit(node):
    color[node] = VISITING; position[node] = len(active); active.push(node)
    for next in neighbors[node]:
        if color[next] == VISITING: return active[position[next]:] + [next]
        if color[next] == UNSEEN:
            cycle = visit(next); if cycle: return cycle
    active.pop(); position.remove(node); color[node] = DONE; return []
# scan every unseen node; any returned cycle blocks completion
```

Trace moment	Colors / active path	Conclusion
Enter 0, 1, 2, 3	0, 1, 2, 3 are visiting	Current path is 0 -> 1 -> 2 -> 3
Inspect 3 -> 1	1 is still visiting	Back edge proves cycle 1 -> 2 -> 3 -> 1
Return	No need to finish remaining branches	Not every dependency can complete

NEAR MISS

Why one visited set fails
It cannot distinguish completed work from a node still on the active path.

COMPARISON

Valid alternative
Kahn's BFS removes zero-indegree nodes. If it removes fewer than V, a cycle remains.

COMPLEXITY

Every node changes color at most twice and every directed edge is inspected once: O(V + E) time and O(V) color, active-path, position, and DFS-stack space.

PROBLEM 4 / ATTEMPT FIRST

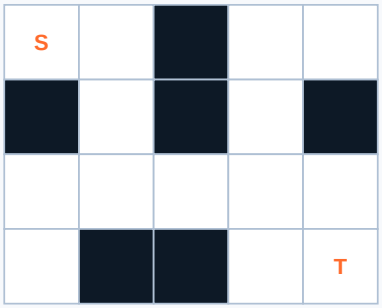
Return one shortest route through a grid

Four-direction moves have equal cost. The learner must return the route itself, not only prove reachability or report its length.

PROMPT

In the 4 x 5 grid, move through open cells using Right, Down, Left, Up as the fixed neighbor order. Return any shortest coordinate path from S to T, [S] when source equals target, and [] when unreachable.

OPEN CELLS ARE WHITE; # CELLS ARE BLOCKED



STABLE PARENT

Discovery contract

The first time a cell is discovered, save exactly one parent and do not overwrite it.

RECONSTRUCTION

Answer contract

Backtrack from T to S through parent links, then reverse the collected coordinates.

Before coding

- [ ] Separate queue order, discovered cells, parent links, and the final path.
- [ ] Choose the exact moment a parent is assigned and a cell becomes discovered.
- [ ] State when it is safe to stop: target discovery or target dequeue.
- [ ] Test source equals target, blocked target, unreachable target, one corridor, and multiple shortest paths.
- [ ] Explain why fixed neighbor order stabilizes this example without proving a globally lexicographic path.

Pre-code decision record

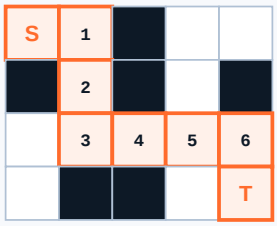
Decision	Write this before implementation	Quality check
Guarantee	Minimum moves plus the coordinate route	Reachability alone is insufficient
Frontier + state	FIFO queue; discovered cells; one stable parent	Keep final path separate
Invariant	Each queued cell has a shortest parent chain from S	Assign parent before enqueue
Attempt trace	Predict the first parent chain that reaches T	Use stated neighbor order
Boundaries	[S] for same cell; [] if blocked or unreachable; tied routes allowed	Never overwrite parents

PROBLEM 4 / WORKED REVIEW

Parent links turn a distance guarantee into a route

BFS provides the minimum-edge guarantee. The parent map preserves the evidence needed to reconstruct one of the shortest paths.

WORKED SHORTEST PATH; NUMBERS SHOW PATH ORDER



01
Discover
Queue S with parent[S] = none.

02
Assign once
Set each unseen neighbor's parent before enqueue.

03
Stop safely
First target discovery has minimum distance.

04
Reverse
Follow T back to S, then reverse the chain.

```
parent = {source: none}; queue = [source]
while queue is not empty and target not in parent:
    cell = queue.pop_left()
    for next in valid_neighbors(cell):
        if next in parent: continue
        parent[next] = cell; queue.push_right(next)
if target not in parent: return []
path = []; cell = target
while cell is not none: path.append(cell); cell = parent[cell]
reverse(path); return path
```

Evidence	Worked result
Path	(0,0) -> (0,1) -> (1,1) -> (2,1) -> (2,2) -> (2,3) -> (2,4) -> (3,4)
Length	7 moves; eight coordinates including source and target
Why shortest	Every discovered cell receives its parent from the earliest reachable BFS layer
Multiple answers	Another equal-length route is valid if its parent chain follows the same first-discovery rule

WHAT FAILS
Near miss
Overwriting a parent destabilizes the chain. DFS finds a route but may not find the fewest moves.

SWITCH METHOD
Prompt change
Different cell costs require a cost-aware shortest-path method, not equal-layer BFS.

COMPLEXITY

Each grid cell is discovered at most once and checks four neighbors: O(R x C) time and O(R x C) queue, parent, and output space.

CROSS-PROBLEM REVIEW

Explain the decision, not only the code

A strong answer links the prompt guarantee to frontier order, state meaning, visited timing, tests, and complexity. The method name alone is not evidence.

Problem	Decisive cue	Traversal	Extra state	Invariant / proof
Components	Finish every disconnected region	DFS or BFS	Global discovered + count	Each unseen outer-loop start completes exactly one component
Minimum edges	Equal-cost shortest distance	BFS	Distance at discovery	Queue order is nondecreasing shortest distance
Dependency cycle	Edge back to active directed path	3-color DFS	Unseen / visiting / done	Visiting nodes are exactly the current recursion path
Shortest route	Minimum edges plus actual path	BFS	Stable parent at discovery	First discovery supplies a shortest parent chain

Use the 0 / 1 / 2 explanation rubric

Score	Evidence standard	What to improve
0 - absent	Method is named after coding, or state meanings never appear	Return to the prompt guarantee and define state before the loop
1 - prompted / late	Correct idea appears after a hint, failed trace, or unexplained change	Move the decision to the cue that should trigger it
2 - independent	Cue, state, invariant, timing, tests, and complexity appear at the correct decision point	Preserve the behavior and test it on an adjacent prompt

WEAK EVIDENCE

Correct but silent

Code returns the expected sample, but the learner cannot say why traversal order and state guarantee correctness.

STRONG EVIDENCE

Interview-ready

The learner predicts the approach, traces the state, tests a boundary, and explains what prompt change would invalidate it.

CORRECTION SENTENCE

When I notice [cue], I will state [required guarantee], define [frontier and state], and name [invariant] before implementation so the choice is observable.

COMPLETION AND NEXT ACTION

Save four decisions and one correction

The task is complete when each attempt has inspectable reasoning. A solved sample without cue, state, proof, tests, and a near miss is not yet useful interview evidence.

Required attempt	Completion proof	Near-miss check
Connected components	Count = 4; outer loop and discovery invariant explained	Minimum-edge version would require BFS
Shortest unweighted path	Distance = 3; queue layers and mark-on-discovery shown	Weighted edges require cost-aware search
Directed dependency cycle	False; cycle 1 -> 2 -> 3 -> 1 proved with visiting state	One visited set is insufficient
Shortest reconstructed route	A 7-move path; stable parent chain reversed	DFS does not guarantee minimum moves

Common mistakes

+ Choosing BFS or DFS from a keyword instead of the required guarantee.
+ Mixing frontier order, visited state, distance or color, parent, and final answer into one vague variable.
+ Marking on dequeue and allowing duplicate work or parent overwrite.
+ Using a single visited set for a directed active-path cycle.
+ Starting from one node and forgetting disconnected components.
+ Claiming O(V + E) without explaining the graph representation and stored edges.

Completion check

- [ ] All four prompts were attempted before the worked reviews were opened.
- [ ] Every solution records cue, traversal, state, invariant, three tests, complexity, and near miss.
- [ ] The component, layer, active-path, and parent-chain traces can be explained aloud.
- [ ] One repeated weak decision and one observable correction sentence are saved.
- [ ] The next practice problem is chosen to test the correction, not to add a new topic.

FINAL TALK TRACK

I choose [method] because the prompt requires [guarantee]. My frontier is [structure]; state means [definition]; invariant is [truth]. I mark discovery [when]. I will test [boundaries]. Complexity is [bound]. I would switch approaches if [prompt change].